

Support Vector Regression

Importing the dataset

```
In [15]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Importing the dataset

```
In [16]: df = pd.read_csv('01_dataset/Position_Salaries.csv')
X = df.iloc[:, 1:-1].values
y = df.iloc[:, -1].values
```

```
In [17]: print(X)
```

```
[[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
[10]]
```

```
In [18]: print(y)
```

```
[ 45000  50000  60000  80000 110000 150000 200000 300000 500000
1000000]
```

```
In [19]: y = y.reshape(len(y), 1)
```

```
In [20]: print(y)
```

```
[[ 45000]
 [ 50000]
 [ 60000]
 [ 80000]
 [110000]
 [150000]
 [200000]
 [300000]
 [500000]
[1000000]]
```

Feature Scaling

```
In [21]: from sklearn.preprocessing import StandardScaler
sc_X = StandardScaler()
sc_y = StandardScaler()
X = sc_X.fit_transform(X)
y = sc_y.fit_transform(y)
```

```
In [22]: print(X)
```

```
[[-1.5666989 ]
 [-1.21854359]
 [-0.87038828]
 [-0.52223297]
 [-0.17407766]
 [ 0.17407766]
 [ 0.52223297]
 [ 0.87038828]
 [ 1.21854359]
 [ 1.5666989 ]]
```

```
In [23]: print(y)
```

```
[[-0.72004253]
 [-0.70243757]
 [-0.66722767]
 [-0.59680786]
 [-0.49117815]
 [-0.35033854]
 [-0.17428902]
 [ 0.17781001]
 [ 0.88200808]
 [ 2.64250325]]
```

Feature Scaling

```
In [10]: from sklearn.preprocessing import StandardScaler
sc_X = StandardScaler()
sc_y = StandardScaler()
x = sc_X.fit_transform(X)
y = sc_y.fit_transform(y)
```

```
In [24]: print(X)
```

```
[[-1.5666989 ]
 [-1.21854359]
 [-0.87038828]
 [-0.52223297]
 [-0.17407766]
 [ 0.17407766]
 [ 0.52223297]
 [ 0.87038828]
 [ 1.21854359]
 [ 1.5666989 ]]
```

```
In [25]: print(y)
```

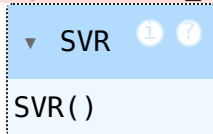
```
[[-0.72004253]
 [-0.70243757]
 [-0.66722767]
 [-0.59680786]
 [-0.49117815]
 [-0.35033854]
 [-0.17428902]
 [ 0.17781001]
 [ 0.88200808]
 [ 2.64250325]]
```

Training the SVR model on the whole dataset

```
In [26]: from sklearn.svm import SVR
regressor = SVR(kernel = 'rbf')
regressor.fit(X, y)
```

```
/home/ochwo-edrian-jude/miniconda3/envs/ds4b/lib/python3.11/site-packages/sklearn/
utils/validation.py:1408: DataConversionWarning: A column-vector y was passed when a
1d array was expected. Please change the shape of y to (n_samples, ), for example using
ravel().
  y = column_or_1d(y, warn=True)
```

Out[26]:



Predicting the new result

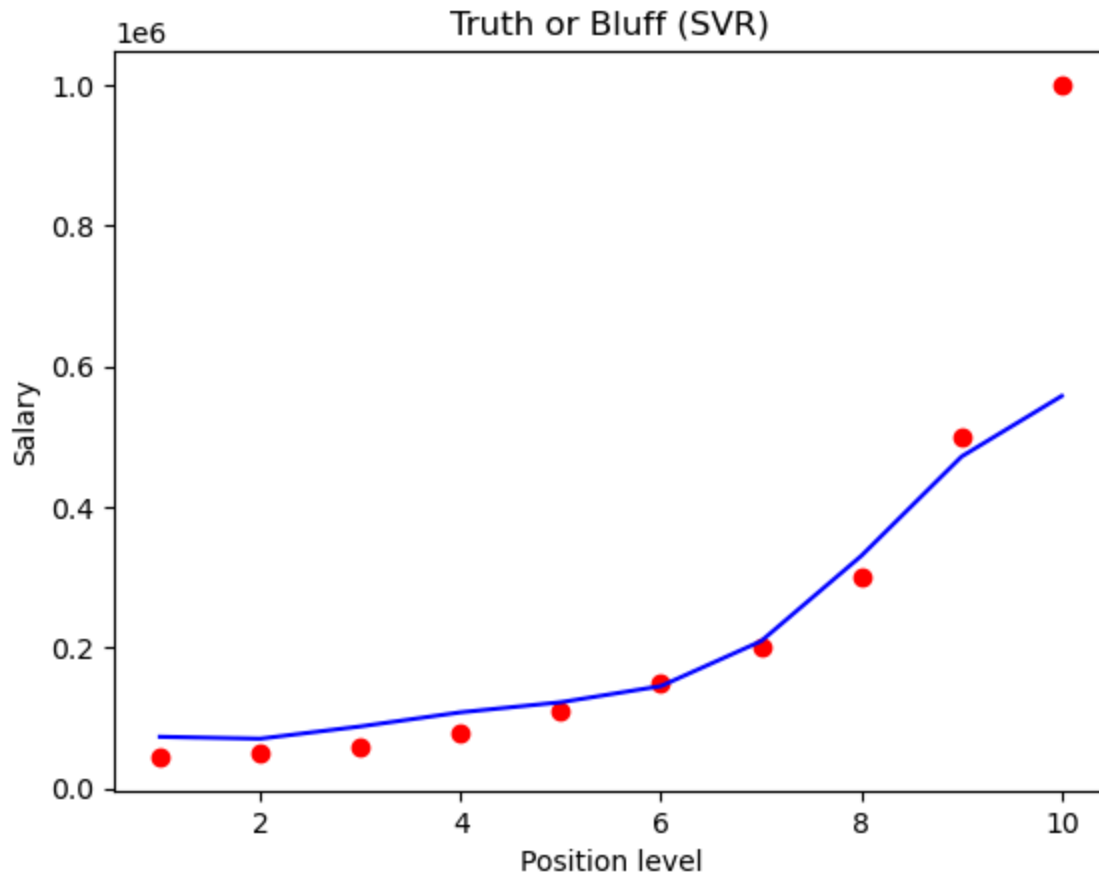
```
In [29]: sc_y.inverse_transform(regressor.predict(sc_X.transform([[6.5]])).reshape(-1,1))

# -- note that the inverse_tranform helps handle the fact that the y values
```

Out[29]: array([[170370.0204065]])

Visualising the SVR results

```
In [36]: plt.scatter(sc_X.inverse_transform(X).flatten(), sc_y.inverse_transform(y).f
plt.plot(sc_X.inverse_transform(X).flatten(), sc_y.inverse_transform(regress
plt.title("Truth or Bluff (SVR)")
plt.xlabel("Position level")
plt.ylabel("Salary")
plt.show()
```

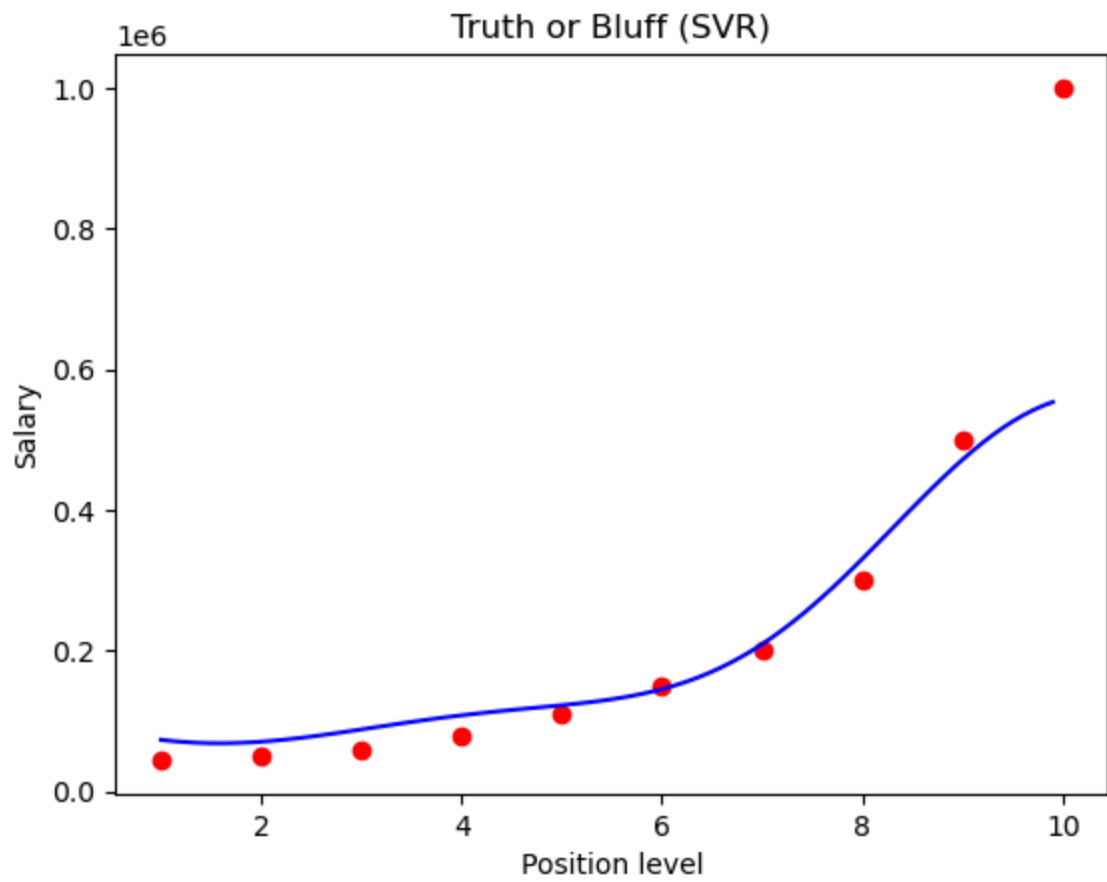


Visualising the SVR results (for higher resolution and smoother curve)

```
In [38]: X_grid = np.arange(min(sc_X.inverse_transform(X)), max(sc_X.inverse_transform(X)), 0.1).flatten()
X_grid = X_grid.reshape((len(X_grid), 1))
plt.scatter(sc_X.inverse_transform(X), sc_y.inverse_transform(y), color = 'r')
plt.plot(X_grid, sc_y.inverse_transform(regressor.predict(sc_X.transform(X_grid))), color = 'b')
plt.title('Truth or Bluff (SVR)')
plt.xlabel('Position level')
plt.ylabel('Salary')
plt.show()
```

/tmp/ipykernel_26952/47483053.py:1: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)

```
X_grid = np.arange(min(sc_X.inverse_transform(X)).flatten(), max(sc_X.inverse_transform(X)).flatten(), 0.1).flatten()
```



In []:

In []:

In []: